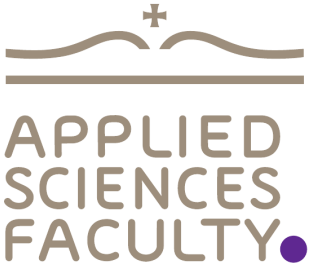
C++ — додаток. Qt. Lecture 4 — діалоги, меню, тощо





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

"Hello, Image"

- Розширимо нашу програму, щоб вона могла переглядати довільні зображення.
- Для цього нам знадобиться, як мінімум, кнопка для завантаження. Можна було б розташувати її на або під "Quit", але це якось несерйозно. Спробуємо розташувати *поруч*.
- Для цього QVBoxLayout вже не підходить. В принципі, можна викрутитися[1], додавши замість Quit (з використанням addWidget) його горизонтального напарника — Q**H**BoxLayout, (використавши addLayout, який є і в QLayout та його нащадків), куди покласти дві кнопки. Але це не дуже гарно. Краще скористатися **QGridLayout** — менеджером розташування, який дозволяє "розкладати" компоненти "сіточкою" — в таблиці.

[1] В деяких фреймворках часто доводиться так робити...

QGridLayout

- Його метод addWidget() дещо складніший — слід вказувати, куди віджет класти. Він існує в двох варіантах:

```
void QGridLayout::addWidget(QWidget * widget,
                            int row, int column,
                            Qt::Alignment alignment = 0)

void QGridLayout::addWidget(QWidget * widget,
                            int fromRow, int fromColumn,
                            int rowSpan, int columnSpan,
                            Qt::Alignment alignment = 0)
```

- Першому варіанту передається вказівник на віджет, який додається[2], та координати в сітці, де його слід розташувати. Номери стовпців та рядків відраховуються від нуля.
- Другий хитріший. Він використовується, щоб розташувати віджет, який має займати декілька клітинок. Для цього, крім координат початку — лівої верхньої клітинки, передаємо скільки клітинок по горизонталі (columnSpan) і по вертикалі (rowSpan) він повинен займати.

[2] Володіння об'єктом передається — надалі ним керуватиме сам layout.

QGridLayout

- Обидва вказані прототипи:

  void QGridLayout::addWidget(QWidget * widget,
  .....
  Qt::Alignment alignment = 0)

  мають ще один цікавий аргумент — **alignment**. Він значить те ж, що розглянуте раніше вирівнювання вмісту QLabel, але стосується розташування компонент-віджетів у комірках QGridLayout.
- Розглянуті раніше QVBoxLayout і QHBoxLayout теж мають такий параметр по замовчуванню. 0 означає заповнювати всю комірку, решта відповідають описаним раніше членам перерахування Qt::Alignment.
- Якщо alignment не дорівнює нулю, менеджер розташування починає "поважати" sizeHint.
- Другий варіант виклику, якщо йому передати rowSpan=-1 або columnSpan=-1, розташовує віджет, займаючи всі клітинки до самого низу чи до правого краю, відповідно.

Створюємо головний віджет

```
MainWidget::MainWidget(){
    label = new QLabel();
    label->setAlignment(Qt::AlignCenter);
    label->setMinimumSize( 400,300 );

    quitButton = new QPushButton(tr("&Quit"));
    loadButton = new QPushButton(tr("&Load"));

    QObject::connect(quitButton, SIGNAL(clicked()), this, SLOT(quit()));
    QObject::connect(loadButton, SIGNAL(clicked()), this, SLOT(load()));

    QGridLayout *layout = new QGridLayout;
    layout->addWidget(label, 0, 0, 1, 2);
    layout->addWidget(loadButton, 1, 0);
    layout->addWidget(quitButton, 1, 1);

    setLayout( layout );

    setWindowTitle(tr("Hello, Image"));
}
```

Нового в коді конструктора вище не дуже багато — створюємо додаткову кнопку "Load", замість вертикального розташування використано QGridLayout. Мітка, на якій будуть картинки, займає два стовпі одного рядка:

```
layout->addWidget(label, 0, 0, 1,  2);
```

Крім того, задаємо мінімальний розмір мітки:

```
label->setMinimumSize( 400,300 );
```

Оголошення класу MainWidget

```
class MainWidget : public QWidget
{
    Q_OBJECT

public:
    MainWidget();
    ~MainWidget();

private slots:
    void quit();
    void load();

protected:
    void resizeEvent(QResizeEvent *event);

private:
    QLabel *label;
    QPushButton *quitButton;
    QPushButton *loadButton;
    QPixmap pixmap1;
};
```

- Картину, яку показуємо, теж зберігаємо окремо:

  QPixmap pixmap1;

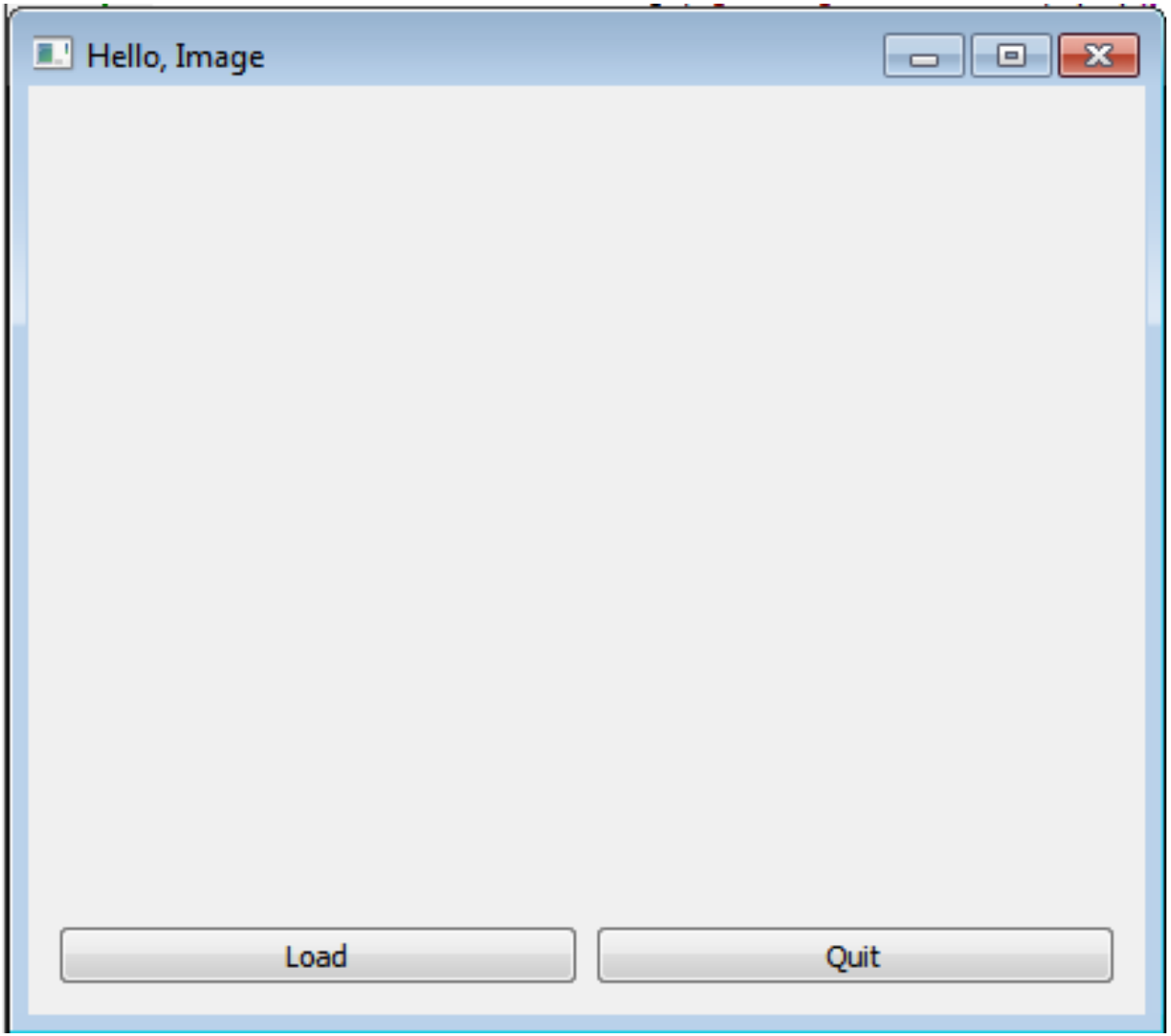
  Зроблено це для полегшення масштабування — при зміні розміру ми виводитимемо вихідне зображення, перемасштабоване на потрібний розмір. Якщо кожен раз масштабувати результат попереднього масштабування, похибки заокруглення дуже швидко знищать зображення.

- Щоб зрозуміти, навіщо це потрібно, поекспериментуйте з наступник кодом :

```
// #define BAD_SCALING
#ifndef BAD_SCALING
        label->setPixmap( pixmap1.scaled(w,h,Qt::KeepAspectRatio) );
#else
        label->setPixmap( label->pixmap()->scaled(w,h,Qt::KeepAspec
#endif
```

  і прикладі "Hello image" до цієї лекції. Зокрема, поспостерігайте, як змінюється картинка під час перемасштабування, якщо використовувати кожен із наведених варіантів коду.

Тепер наша програма виглядатиме так:



Залишається розібратися, як вона завантажує зображення.

```
void MainWidget::load(){
    QString fileName = QFileDialog::getOpenFileName(this,
                tr("Open image file"), "",
                tr("Jpeg files (*.jpg *.jpeg);;
                PNG files (*.png);;
                GIF files (*.gif)"));
    if ( !fileName.isEmpty() ){
        pixmap1.load(fileName);
        if (!pixmap1.isNull() ){
            int w = label->width();
            int h = label->height();

            label->setPixmap( pixmap1.scaled(w,h,Qt::KeepAspectRatio) );
        }else
        {
            QMessageBox::information(this, tr("Hello, Image"),
                                tr("Failed to load %1.").arg(fileName));
            return;
        }
    }
}
```

QFileDialog

- Для того, щоб вибрати файл, скористаємося стандартним діалогом, QFileDialog.
- Цей клас призначений для створення діалогів, котрі дозволяють вибирати файли.
- Найпростіший спосіб ним скористатися — за допомогою статичних функцій. Одна із таких функцій: getOpenFileName — створює діалог, який дозволяє вибрати один файл, що вже існує. Вона повертає ім'я вибраного файлу, або порожню стрічку, якщо натиснуто Cancel. Тому надалі ми і перевіряємо — якщо стрічка не порожня, значить файл вибрали.
- Перший її аргумент — вікно-власник ("батько") діалогу.
- Другий — заголовок вікна діалогу.
- Третій — початковий шлях. Порожня стрічка вказує, починати в поточній директорії програми. Зауважте, потреба перекладати цю стрічку навряд чи виникне, тому tr() не використано!
- Четвертий — стрічка, що описує, які файли можна відкривати — фільтр. Детальніше про неї трішки пізніше.

- Поміж інших статичних функцій для роботи з QFileDialog є:
  - QFileDialog::getExistingDirectory — вибрати директорію, що існує.
  - QFileDialog::getOpenFileNames — вибрати один або декілька файлів, що вже існують.
  - QFileDialog::getSaveFileName — ім'я файлу, який не обов'язково існує.
- Працювати з діалогом можна й безпосередньо — створюючи та налаштовуючи відповідний об'єкт. Детальніше див. `http://qt-project.org/doc/qt-5/qfiledialog.html`
- Інші корисні діалоги:
  - QProgressDialog — власне, прогрес тривалих процесів.
  - QMessageBox — вже розглядався, служить для інформування користувача та отримання від нього відповідей на прості запитання.
  - QInputDialog — діалог для отримання одного значення (числа, стрічки, вибраного елементу списку), від користувача.
  - QFontDialog — діалог вибору шрифтів.
  - QErrorMessage — діалог для виведення повідомлень про помилки.
  - QColorDialog — діалог вибору кольорів.
  - Цікавим є і ще один нащадок QDialog, QWizard — клас для створення "візардів", помічників.
- Звичайно, можна створювати і свої діалоги.

Стрічка — фільтр

- Список типів файлів, які можна відкривати, задається стрічкою—фільтром ось такого виду:

  ```
  "Jpeg files (*.jpg *.jpeg);;PNG files (*.png);;\
  GIF files (*.gif)"
  ```

- Формат її менш-більш очевидний:
  - Спочатку йде текстовий опис конкретного фільтра, наприклад: "Jpeg files".
  - Далі, в дужках — маска файлу: "(*.png)". Якщо масок декілька — вони розділені пробілами: "(*.jpg *.jpeg)".
  - Фільтри відділені один від одного **двома** крапками з комою.

- Якщо користувач вибрав якийсь файл (вибір файлу, який не існує, присіче сам QFileDialog), пробуємо його відкрити:

```
if ( !fileName.isEmpty() )
    {
        pixmap1.load(fileName);
```

- Якщо вдалося:

```
if (!pixmap1.isNull() ){
```

- Масштабуємо зображення до розміру мітки, із збереженням пропорцій:

```
int w = label->width();
int h = label->height();
label->setPixmap( pixmap1.scaled(w,h,Qt::KeepAspectRatio) );
```

Метод QPixmap::scaled повертає масштабовану копію, яку ми і кладемо на мітку.

- Якщо ж не вдалося, повідомляємо про це (зауважте використання засобів підстановки QString):

```
}else{
    QMessageBox::information(this, tr("Hello, Image"),
                             tr("Failed to load %1.").arg(fileName));
    return;
}
```

Resizing

- Однак, якщо залишити програму так, при зміні розміру вікна, зображення залишатиметься таким, як було. Негарно.
- Звідки програма знає, що розмір її вікна змінили? Точніше, звідки кожне із вікон, яке це зачепило, знатиме?
- QWidget має захищений віртуальний метод resizeEvent, який викликається при зміні розміру:

  void QWidget::resizeEvent (QResizeEvent * event)

- Тому, реагувати на зміну розміру можна, поступивши класичним для ООП способом — перевизначивши цей віртуальний метод.
- Коли викликається цей метод, розмір **вже** змінено.
- Інформація про старий та новий розмір міститься в переданому об'єкті типу QResizeEvent. Він має два актуальних для нас методи:

  const QSize & QResizeEvent::oldSize () **const**

  який дозволяє подивитися старий розмір, та

  const QSize & QResizeEvent::size () **const**

  котрий повертає новий. В принципі, те ж саме значення, яке поверне і size() об'єкта, чий розмір змінився.

Далі все просто:

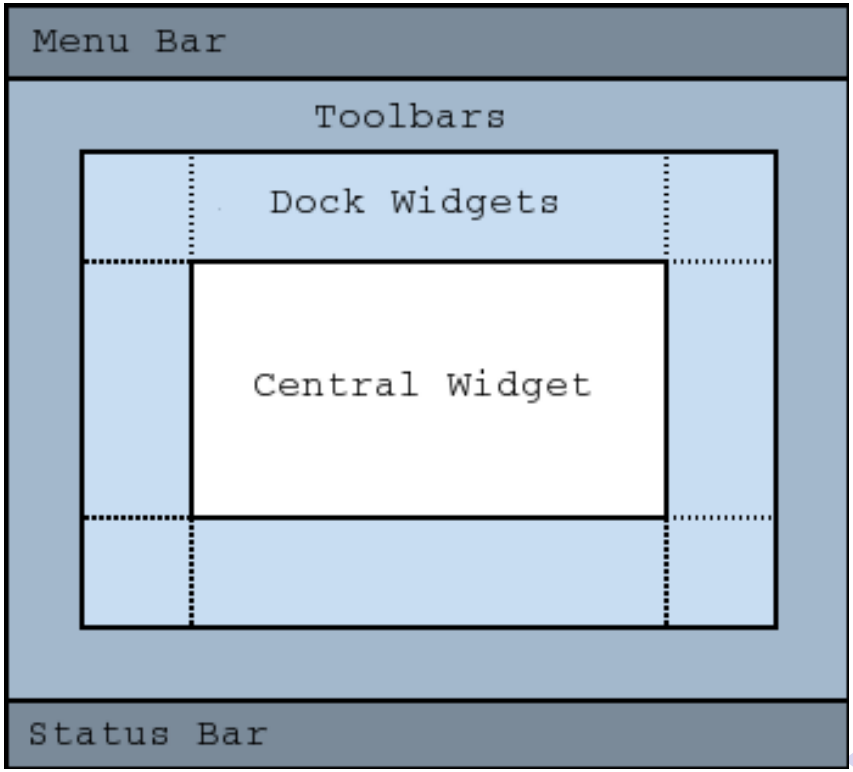
```
void MainWidget::resizeEvent(QResizeEvent *event)
{
    QWidget::resizeEvent(event);

    double w_ration = static_cast<double>(
        event->size().height())/event->oldSize().height();
    double h_ration = static_cast<double>(
        event->size().width())/event->oldSize().width();
    int w = label->width()*w_ration;
    int h = label->height()*h_ration;
    if( label->pixmap() )
    {
        label->setPixmap(
            pixmap1->scaled(w,h,Qt::KeepAspectRatio) );
    }
}
```

Зверніть увагу на виклик відповідного методу базового класу, щоб він теж зміг відпрацювати.

Програма із звичним виглядом — меню, стрічкою статусу, тощо

- Побудована нами програма, хоча вже цілком виконує свої функції, виглядає нетипово.
- Більшість програм мають меню, toolbar-и, statusbar-и і т.д. Все це можна створити самостійно, повісивши їх на QWidget. Однак, навіщо винаходити велосипед?
- QMainWindow — нащадок QWidget, який вже "розмічено" для додавання всіх названих компонент, представлених класами QToolBars, QMenuBar, QStatusBar та QDockWidgets.



QAction

- Перш ніж перейти до модифікації програми, слід розглянути ще одну важливу річ. В будь-якій нетривіальній програмі ту саму команду можна віддати різними способами — за допомогою меню, тулбарів, кнопок, шоткатів.
- Для спрощення роботи з цими сутностями в Qt існує поняття "дії" — action, реалізованої відповідним класом QAction.
- QAction може містити текст опису, іконку, шоткат для виклику, статус, опис, який випливає, коли затримати курсор над відповідним віджетом (tooltip), і текст "Що це?".
- Як би дія не буда "активована", генерується її сигнал triggered(), який можна прив'язати до слота, що реалізовуватиме реакцію на неї:

```
QObject::connect(loadAction, SIGNAL(triggered()), this, SLOT(load()));
```

- "Дію", у свою чергу, можна додавати в меню:

```
fileMenu->addAction(loadAction);
```

до тулбара:

```
fileToolBar->addAction(loadAction);
```

і т. д.

Оголошення нашого MainWidget[3]

```
class MainWidget : public QMainWindow{
        .....
private slots :
    void quit ();
    void load ();
    void about();
        .....
    QAction *loadAction;
    QAction *quitAction;
    QAction *aboutAction;

    QMenu *fileMenu;
    QMenu *helpMenu;

    QToolBar *fileToolBar;
    QToolBar *helpToolBar;

    QLabel *sbImageWidth, *sbImageHeigth;
    QLabel *sbImageName;
};
```

[3]Так як все оголошення тут не поміститься, показую лише нове. Повний код див. у прикладах.

Основні зміни відбулися в створенні вікна

- Щоб центральний віджет – мітку, було краще видно, виділимо її кольором:

```
label->setAutoFillBackground(true);
label->setStyleSheet("QLabel { background-color : grey }");
```

- Для цього використано метод setStyleSheet з QWidget, який встановлює стиль для відображення. Синтаксис опису можна подивитися тут: `http://qt-project.org/doc/qt-5/stylesheet.html`, він достатньо очевидний.
- Встановлюється центральний віджет головного вікна методом setCentralWidget():

```
setCentralWidget( label );
```

Дії

- Замість кнопок, створюємо дії:

```
loadAction = new QAction(tr("&Open"), this);
quitAction = new QAction(tr("&Quit"), this);
aboutAction = new QAction(tr("&About"), this);
```

- Користуючись нагодою, присвоюємо їм іконки:

```
loadAction->setIcon( QIcon("./load.png"));
quitAction->setIcon( QIcon("./quit.png"));
aboutAction->setIcon( QIcon("./about.png"));
```

Ікони "надаються" класом QIcon. Детальніше про нього див. документацію. Важливо, що коли робити так, як в коді вище, файли іконок повинні бути поруч із програмою, інакше вона їх не знайде.

- Нарешті, прив'язуємо їх до відповідних слотів:

```
QObject::connect(loadAction, SIGNAL(triggered()), this, SLOT(load()));
QObject::connect(quitAction, SIGNAL(triggered()), this, SLOT(quit()));
QObject::connect(aboutAction, SIGNAL(triggered()), this, SLOT(about()))
```

Меню

- Одне випадне меню реалізовується об'єктом класу QMenu:

```
fileMenu = menuBar()->addMenu(tr("&File"));
```

- Метод QMainWindow::menuBar() повертає екземпляр QMenuBar для поточного вікна. Додати нове меню до нього можна методом addMenu().
- Йому передається назва відповідного меню, із можливим шоткатом.
- Додавати до створеного випадного меню елементи можна методом addAction:

```
fileMenu->addAction(loadAction);
```

- Роздільники між пунктами вставляє метод addSeparator():

```
fileMenu->addSeparator();
```

Toolbar

- Окрім меню, більшість програм надають панелі інструментів, які містять кнопки для найчастіше потрібних операцій.
- В Qt за них відповідає клас QToolBar.
- Метод addToolBar класу QMainWindow, якому передається текстова стрічка[4], створює ToolBar, вставляє його у відповідну область вікна, та повертає вказівник на щойно створену панель інструментів.
- Додати дію до панелі можна методом addAction(). При цьому буде використано іконку дії, а якщо її немає — текстовий опис. Сепаратор додається addSeparator().
- Все разом виглядає так:

```
fileToolBar = addToolBar(tr("File"));
fileToolBar->addAction(loadAction);
fileToolBar->addSeparator();
fileToolBar->addAction(quitAction);

helpToolBar = addToolBar(tr("Help"));
helpToolBar->addAction(aboutAction);
```

[4] Є й інші його варіанти — див. документацію.

Стрічка статусу — 1

- Панель статусу реалізовується класом QStatusBar.
- Метод statusBar() класу QMainWindow повертає вказівник на поточну стрічку статусу, створюючи її за першого виклику.
- Стрічка статусу вміє показувати інформацію трьома різними способами:
  - Тимчасові повідомлення. Виводяться методом showMessage():

    ```
    statusBar()->showMessage(tr("Ready"));
    ```

    Очищається воно слотом clearMessage(). Можна також встановити час існування.
  - Нормальні повідомлення (addWidget).
  - Перманентні повідомлення (addPermanentWidget).
- І нормальні і перманентні повідомлення відображаються маленькими віджетами (QLabel, QProgressBar чи й QToolButton). Різниця між ними в тому, що нормальні можуть бути перекриті тимчасовим повідомленням, перманентні — ні, крім того, нормальні додаються з лівого краю, перманентні – з правого.
- Додаються вони методом addWidget().

Стрічка статусу — 2

- В програмі передбачено 4 таких віджета: ім'я файлу, його ширина, роздільник із літерою "X" та висота файлу:

```
sbImageWidth = new QLabel("     ");
sbImageWidth->setMinimumSize( sbImageWidth->sizeHint() );
sbImageWidth->setAlignment(Qt::AlignCenter);
sbImageHeigth = new QLabel("     ");
sbImageHeigth->setMinimumSize( sbImageHeigth->sizeHint() );
sbImageHeigth->setAlignment(Qt::AlignCenter);

sbImageName = new QLabel();
sbImageName->setAlignment(Qt::AlignLeft);

statusBar()->addWidget(sbImageName,1);
statusBar()->addPermanentWidget(sbImageWidth);
statusBar()->addPermanentWidget( new QLabel("X"));
statusBar()->addPermanentWidget(sbImageHeigth);
```

- Щоб мітки мали адекватну ширину, заповнюємо їх кількома пробілами (п'ятьма, оцінюючи ймовірну кількість цифр), і вказуємо цю ширину як мінімальну. Метод, звичайно, примітивний, у серйозній програмі так робити не варто.

Довідкова інформація

На цьому створення головного вікна програми завершено. Перш ніж перейти до нюансів обробки подій, наведу посилання на документацію, де описано використані вище класи:

- `http://qt-project.org/doc/qt-5/qmainwindow.html` — клас головного вікна програми.
- `http://qt-project.org/doc/qt-5/qaction.html` — дії.
- `http://qt-project.org/doc/qt-5/stylesheet.html` — опис Qt Style Sheets.
- `http://qt-project.org/doc/qt-5/qicon.html` — клас ікон.
- `http://qt-project.org/doc/qt-5/qmenu.html` — випадне меню, для використання в головному меню, контекстних меню, тощо.
- `http://qt-project.org/doc/qt-5/qmenubar.html` — стрічка меню головного вікна.
- `http://qt-project.org/doc/qt-5/qstatusbar.html` — стрічка статусу.
- `http://qt-project.org/doc/qt-5/qtoolbar.html` — панель інструментів.

About

Для виведення інформації About скористаємося все тією ж QLabel. Але тепер створимо відповідний об'єкт та "заселимо" його інформацією:

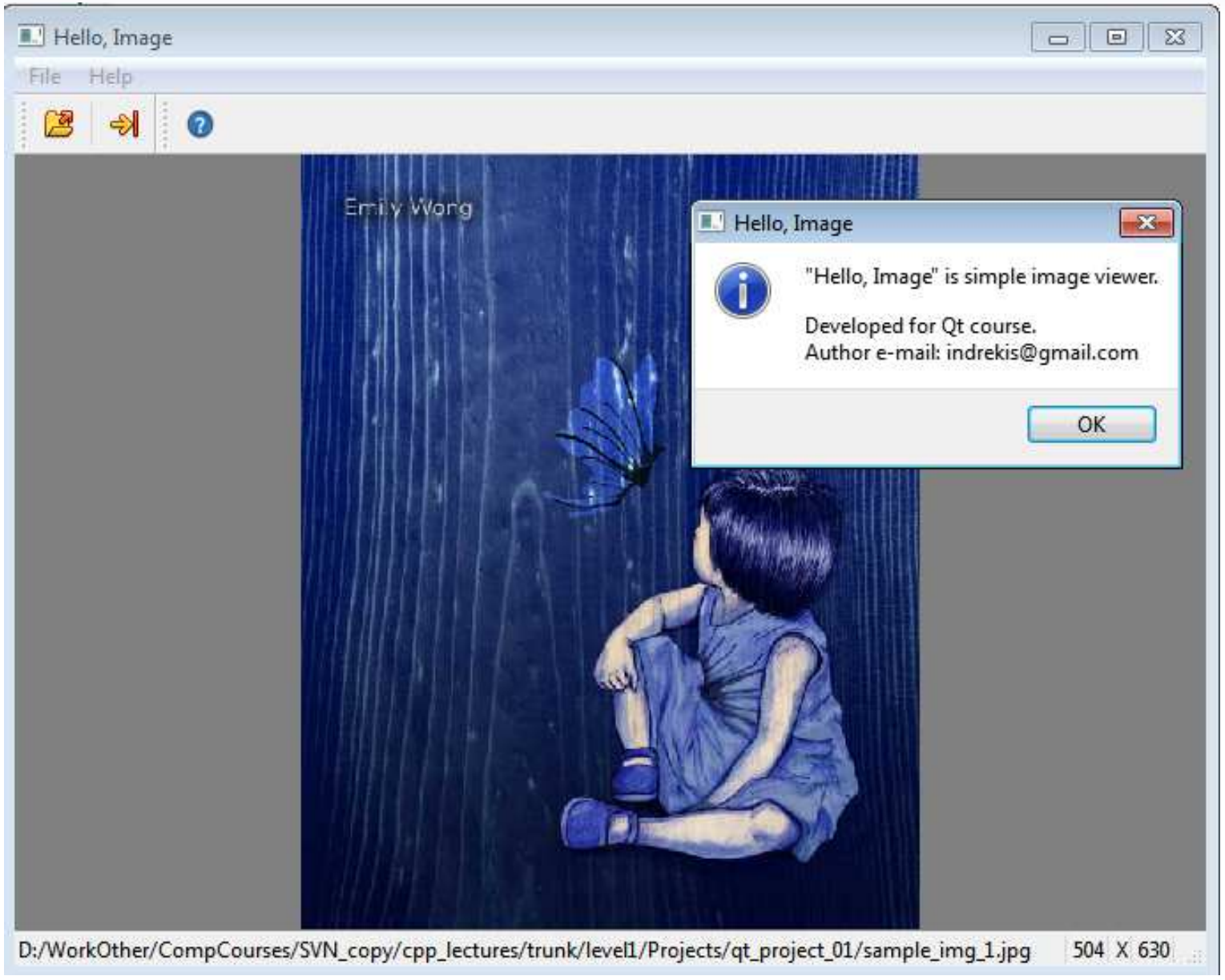
```
void MainWidget::about()
{
    QMessageBox aboutBox;
    aboutBox.setIcon(QMessageBox::Information);
    aboutBox.setWindowTitle(tr("Hello, Image"));
    aboutBox.setText(tr("\"Hello, Image\" is simple image viewer."));
    aboutBox.setInformativeText(tr("Developed for Qt course. \n \
Author e-mail: indrekis@gmail.com"));
    aboutBox.setStandardButtons(QMessageBox::Ok);
    aboutBox.exec();
}
```

Також можна скористатися QMessageBox::about із статичного API, хоча вона менш гнучка.

Завантаження зображень

Слот load тепер повинен також оновлювати стрічку статусу:

```
label->setPixmap( pixmap1->scaled(w,h,Qt::KeepAspectRatio) );
// Нове: після успішного завантаження виводимо
// інформацію про файл із зображенням
sbImageName->setText(fileName);
sbImageWidth->setNum( pixmap1->size().width() );
sbImageHeigth->setNum( pixmap1->size().height() );
```


Hello, Image
File Help
Emily Wong
Hello, Image
"Hello, Image" is simple image viewer.
Developed for Qt course.
Author e-mail: indrekis@gmail.com
OK
D:/WorkOther/CompCourses/SVN_copy/cpp_lectures/trunk/level1/Projects/qt_project_01/sample_img_1.jpg
504 X 630

- Наша програма має певний недолік — файли іконок слід розташовувати разом із нею. Це незручно.
- Більшість операційних систем передбачено можливість вбудовувати такі дрібні об'єкти, як іконки, в саму програму.
- Для цього використовується система **ресурсів**.
- Опис її реалізації в Qt ось тут: http://qt-project.org/doc/qt-5/resources.html
- Ми коротко розглянемо, як з нею працювати, на прикладі іконок нашої програми.

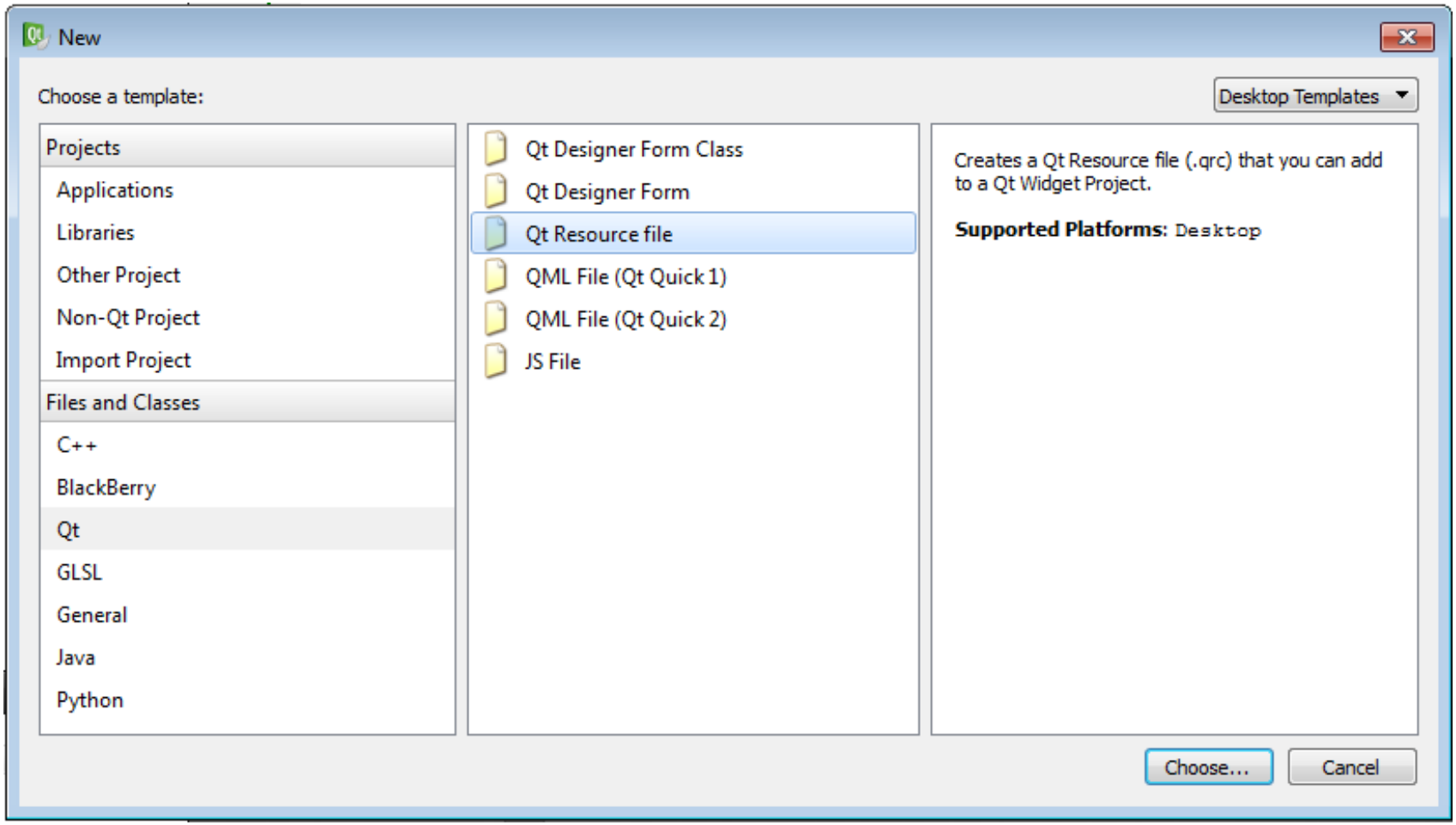
Файл ресурсів — .qrc

Опис ресурсів програми здійснюється в спеціальному файлі, з розширенням .qrc, який є звичайним текстовим файлом у форматі XML

```
<RCC>
    <qresource  prefix="/">
        <file>about.png</file>
        <file>load.png</file>
        <file>quit.png</file>
    </qresource>
</RCC>
```

В ньому перераховано файли із зображеннями. Можна задавати шлях до них відносно шляху, де є .qrc, наприклад так: `images/icon.png`, однак тільки в піддиректоріях тієї директорії, де знаходиться файл .qrc. Детальніше про формат — див. документацію.

Додати до проекту його можна за допомогою File->New file or Project:



Редагувати теж цілком зручно візуально, засобами середовища.

Доступ до ресурсів

По замовчуванню доступ ресурсів можливий за тим же шляхом та іменем, що прописано в файлі ресурсів (.qrc), з двокрапкою на початку:

```
loadAction->setIcon( QIcon(":/load.png"));
quitAction->setIcon( QIcon(":/quit.png"));
aboutAction->setIcon( QIcon(":/about.png"));
```

Це можна змінити, використавши атрибут alias тегу file:

```
<file   alias ="newname.png">images/oldname.png</file>
```

Завдяки цьому атрибуту можна буде писати

```
someAction->setIcon( QIcon(":/newname.png"));
```

замість:

```
someAction->setIcon( QIcon(":/images/oldname.png"));
```

Переклад ресурсів

- Якщо різним мовам потрібні будуть різні ресурси, їх можна вказати за допомогою атрибуту lang тегу qresource:

```
<qresource>
    <file>open.png</file>
</qresource>
<qresource lang="uk">
    <file  alias ="cut.jpg">open_uk.png</file>
</qresource>
```

Зовнішні та вкомпільовані ресурси

- Файл ресурсів, доданий середовищем, по замовчуванню вписується в файл проекту, .pro, так:

```
RESOURCES += \
    hi_res.qrc
```

  При цьому система побудови, qmake, вкомпільовує його в виконавчий файл програми.
- Альтернативний підхід — зовнішні ресурси. Це зручно, наприклад, коли є потреба їх замінювати без перекомпіляції.
- Для цього файл ресурсів слід скомпілювати компілятором ресурсів rcc:

```
rcc -binary hi_res.qrc -o hi_res.rcc
```

- Отриманий двійковий файл ресурсів, hi_res.rcc, підключається так:

```
QResource::registerResource("/path/hi_res.rcc");
```

Ресурси в бібліотеках — 1

- Якщо ресурси використовуються в бібліотеці, код, який буде їх використовувати, повинен спершу їх ініціалізувати за допомогою Q_INIT_RESOURCE(), якій передається базове ім'я файлу ресурсів:

```
CoolLibWidget::CoolLibWidget() : CoolLibBaseWidget()
{
    Q_INIT_RESOURCE(resources);

    QIcon icon1(":/icon1.png");
     ...
}
```

Це гарантує, що ресурси буде прилінковано до програми у випадок статичного компонування.

Ресурси в бібліотеках — 2

- Якщо бібліотека експортує ресурси для програми, ініціалізувати їх повинна сама програма:

```
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    Q_INIT_RESOURCE(coollibres);
```

- За звільнення ресурсів відповідає Q_CLEANUP_RESOURCE().